

An Edge-Centric IoT System for Real-time Crowd Density Estimation, Optical Flow Analysis, and Autonomous Risk Actuation

1st Rohan C Bharadwaj
Department of Computer Science
R.V. College of Engineering
Bengaluru, India
rohanbharadwaj.cs24@rvce.edu.in

2nd Sathish Dath D S
Department of Computer Science
R.V. College of Engineering
Bengaluru, India
sathishdathds.cs24@rvce.edu.in

3rd Ramakrishna V Bhat
Department of Computer Science
R.V. College of Engineering
Bengaluru, India
ramakrishnavbhat.cs24@rvce.edu.in

Abstract—Real-time crowd safety management is a critical priority for public venues to prevent crowd crush events, stampedes, and structural failures. Traditional video-surveillance systems rely on centralized cloud processing, which suffers from network latency, bandwidth overhead, and a lack of fail-safe local actuation. This paper presents an edge-centric, closed-loop Cyber-Physical System (CPS) for crowd safety, featuring a pluggable detection backend that dynamically balances performance and resource constraints. The edge node executes NVIDIA PeopleNet (ResNet-34 ONNX) with vectorized DetectNet_v2 grid decoding as the primary detector, with a YOLOv8 fallback. Centroids are clustered using DBSCAN to evaluate spatial density and Fruin Level-of-Service (LoS) zones. Concurrently, dynamic motion patterns—velocity, compression (negative divergence), and directional chaos—are extracted via Farneback dense optical flow. A predictive risk engine combines these spatial and temporal indicators to calculate a Composite Risk Index (R), applying Exponential Moving Average (EMA) smoothing, hysteresis, and time-based cooldown. An ordinary least squares (OLS) linear regression model is fitted over a sliding window to forecast the Time-to-Critical (TTC) threshold. Closed-loop warning and closure commands are dispatched via local MQTT to an ESP32 actuator node driving a gate servo, status LED blink patterns, and a buzzer. Experimental evaluations on local CUDA edge hardware demonstrate an average inference rate of 6.1 FPS and an end-to-end edge-to-actuation latency under 320 ms, validating the viability of autonomous edge controllers for crowd safety.

Index Terms—Edge Computing, Crowd Safety, Internet of Things, PeopleNet, Optical Flow, DBSCAN, Closed-loop Actuation.

I. INTRODUCTION

Crowd safety management in large-scale public venues, such as transit hubs, concert halls, and sports stadiums, represents a critical engineering challenge. In high-density settings, sudden localized rushes, bottlenecks, and bidirectional flows can rapidly escalate into crowd crush events, causing severe injuries and fatalities. To prevent these disasters, it is imperative to monitor crowd density and movement pattern metrics continuously, evaluating risks in real time to trigger localized

physical safety barriers and audible warnings before density exceeds critical structural thresholds.

Traditional computer vision approaches to crowd counting and flow analysis typically rely on streaming high-resolution video feeds to centralized cloud servers or high-performance GPU workstations. However, this centralized paradigm faces severe operational limitations when deployed in field environments. First, streaming raw video feeds over wireless networks consumes significant bandwidth and exposes the system to network outages. Second, the round-trip latency associated with cloud transmission prevents prompt, closed-loop actuation. For instance, in crowd disasters, the transition from orderly flow to a crush state can occur within seconds, rendering high-latency warnings ineffective. Thus, localized, low-latency edge computing systems are needed to process visual telemetry locally and actuate crowd control barriers autonomously.

To address these limitations, we propose the *Intelligent Crowd Flow & Density Safety Oracle*, an end-to-end edge-centric IoT system that provides decentralized, low-latency, and autonomous crowd control. The proposed system features a pluggable detection backend designed to support NVIDIA PeopleNet (loaded via ONNX Runtime) and YOLOv8 (Ultralytics). The edge node captures video feeds, decodes detections, maps spatial density using DBSCAN clustering, and extracts crowd motion dynamics via dense Farneback optical flow. Rather than relying on static count thresholds, the system evaluates a composite risk score that accounts for density, speed, spatial compression, and directional chaos, predicting the Time-to-Critical (TTC) congestion threshold.

Our system closes the loop by connecting the edge processor to a physical actuator node based on the ESP32 DevKit. Acting as an autonomous safety gate, the ESP32 subscribes to the edge node's risk-driven actuation commands via a localized MQTT broker. It drives a micro servo motor to control a physical barrier, activates warning LEDs in distinct blink modes, and sounds a buzzer to deter further entry into congested corridors. An operator dashboard, implemented in React, connects to the local broker over WebSockets, visualiz-

ing metrics, flow directions, heatmaps, and actuation histories without transmitting raw video feeds outside the local network.

The primary contributions of this paper are summarized as follows:

- We design and implement a closed-loop, edge-centric IoT system architecture that integrates real-time computer vision inference with physical hardware control to mitigate crowd safety hazards locally.
- We present a pluggable detection facade supporting NVIDIA PeopleNet and YOLOv8, detailing the vectorized DetectNet_v2 box-decoding mathematics required to run ResNet-34 models efficiently on edge nodes.
- We formulate a predictive risk model that combines static spatial clustering (DBSCAN) with dynamic crowd motion vectors (Farneback optical flow magnitude, divergence, and angular chaos) to forecast Time-to-Critical (TTC) thresholds using linear regression over a sliding window, incorporating EMA smoothing, hysteresis, and debouncing.
- We construct and evaluate the physical prototype on off-the-shelf edge hardware, demonstrating an end-to-end edge-to-actuation response latency of less than 320 ms and an inference throughput of 6.1 FPS.

The remainder of this paper is structured as follows. Section II reviews related literature in crowd monitoring and edge-based IoT systems. Section III presents the overall system architecture. Section IV describes the proposed methodology, including detection, density, flow, and risk calculations. Section V details the experimental evaluation and results, and Section VI concludes the paper.

II. RELATED WORK

Research in crowd monitoring has transitioned from manual count estimation to advanced computer vision techniques. In this section, we review related literature in computer vision-based crowd analysis, edge computing applications, and closed-loop IoT safety controllers.

A. Crowd Density Estimation and Counting

Modern computer vision techniques for crowd counting mostly employ regression-based density map estimation or object detection. Deep convolutional neural networks (CNNs), such as Multi-Column CNN (MCNN) [1] and Congested Scene Recognition Network (CSRNet) [2], estimate pixel-wise density maps to count individuals in highly congested scenes. However, these models require high-end GPU accelerators for real-time inference, making them unsuitable for deployment on low-cost edge microprocessors. Alternatively, object detection models like YOLO (You Only Look Once) detect individual bounding boxes, which allows spatial coordinates to be extracted directly. While YOLOv8 is highly accurate, its base configurations remain computationally expensive for edge CPUs. In this work, we integrate a pluggable facade that supports a GPU/CPU-optimized NVIDIA PeopleNet detector and a YOLOv8-tiny variant to achieve real-time person detection.

B. Crowd Motion Analysis and Optical Flow

Static density alone is insufficient for predicting crowd crush events; crowd dynamics (velocity, pressure, and chaos) play a vital role. Researchers have applied optical flow methods to analyze motion patterns. Farneback dense optical flow [3] calculates displacement vectors for all pixels in a scene, allowing the computation of velocity fields. Prior studies have used divergence to monitor flow compression at bottleneck areas, identifying convergence points where crowd pressures rise [4]. Furthermore, direction entropy and standard deviation (chaos) have been used as indicators of panic and turbulent motion. In our work, we incorporate these vector-field metrics directly into a composite risk engine, converting raw pixel motion into structured safety metrics at the edge.

C. Edge Computing and Closed-Loop IoT Actuation

Edge computing has emerged as a key paradigm for low-latency IoT applications [5]. By shifting computing resources close to data sources, edge systems reduce bandwidth demand and latency. While edge networks have been used for surveillance, few implement closed-loop physical actuation for crowd safety. Most existing edge architectures operate as passive telemetry nodes, transmitting alerts to human operators in a cloud control center [6]. This paper presents a fully autonomous closed-loop system where edge-calculated risk indices directly drive physical actuators (servo-driven gates and alarms) locally, ensuring safety intervention even during cloud connectivity outages.

III. SYSTEM ARCHITECTURE

The architecture of the Intelligent Crowd Flow & Density Safety Oracle is designed to run on a local network, bypassing cloud servers to minimize latency and guarantee fail-safe operation. The system consists of three hardware components: the Edge Inference Node, the Actuator Node, and the Visual Control Center. Communication is facilitated by a local MQTT broker running on the edge node. Fig. 1 provides a graphical depiction of the component division and system architecture.

The Edge Inference Node consists of a local PC or Raspberry Pi connected to a webcam. The processor runs a multi-threaded Python pipeline that handles frame capture, inference, risk evaluation, and MQTT publishing. The Actuator Node is built on an ESP32 DevKit microcontroller, which connects to the local network over Wi-Fi, subscribes to the actuation topic, and controls a physical micro servo motor, an active buzzer, and a single status LED. The Visual Control Center is a React web application running on a laptop, connecting to the MQTT broker's WebSockets listener to display visual telemetry.

IV. PROPOSED METHODOLOGY

This section details the analytical pipeline deployed on the Edge Inference Node, outlining the mathematical formulations and algorithms used to compute crowd metrics and evaluate risk.

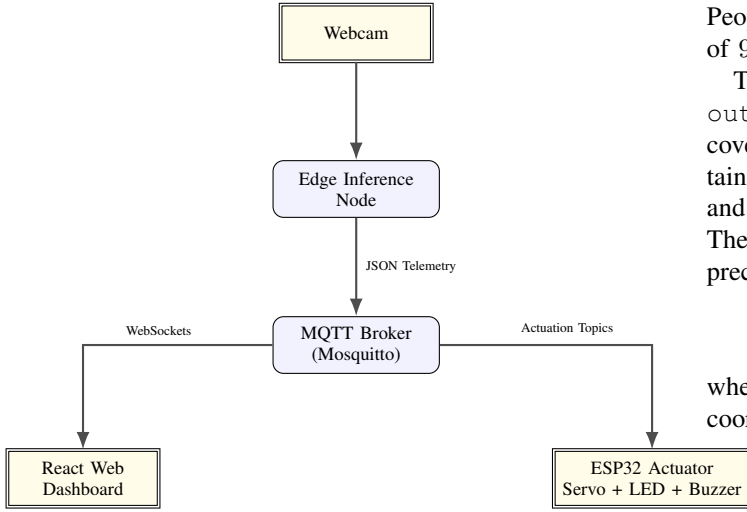


Fig. 1. Decentralized system architecture block diagram.

A. Analytical Pipeline Overview

The software execution sequence follows a strict linear pathway, as illustrated in the flowchart in Fig. 2. Frames are ingested, processed, and converted into actuation directives locally.

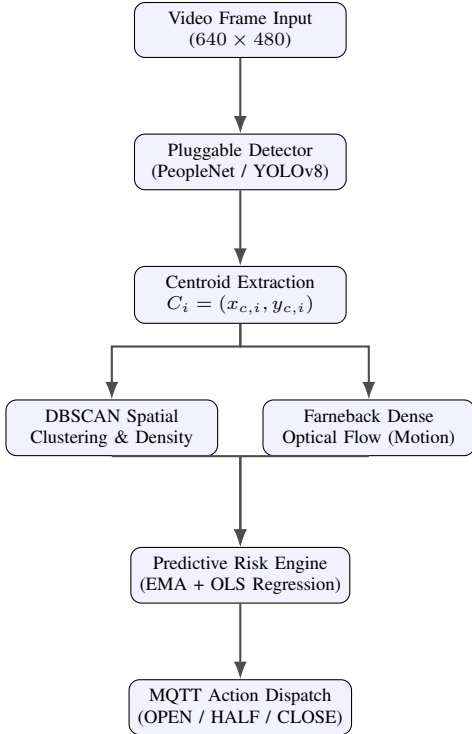


Fig. 2. TikZ flowchart of the localized computer vision and risk pipeline.

B. Pluggable Detector and Vectorized Box Decoding

The `peoplenet-integration` branch introduces a pluggable detector facade. The primary backend is NVIDIA

PeopleNet (ResNet-34 ONNX) configured with an input size of 960×544 pixels. The fallback is YOLOv8s.

The PeopleNet ONNX model produces two output tensors: `output_cov` of shape $(1, 3, 34, 60)$ representing grid cell coverages, and `output_bbox` of shape $(1, 12, 34, 60)$ containing raw bounding box offsets. The grid width is $W_g = 60$ and height is $H_g = 34$, with a cell stride of $S = 16$. The normalized grid center coordinates for cell (r, c) are precomputed:

$$GC_W(c) = \frac{c \cdot S + 0.5}{35.0}, \quad GC_H(r) = \frac{r \cdot S + 0.5}{35.0} \quad (1)$$

where 35.0 is the scale factor constant. The bounding box coordinates in pixel space are decoded via:

$$x_1 = (o_1 - GC_W(c)) \times (-35.0) \times s_x \quad (2)$$

$$y_1 = (o_2 - GC_H(r)) \times (-35.0) \times s_y \quad (3)$$

$$x_2 = (o_3 + GC_W(c)) \times 35.0 \times s_x \quad (4)$$

$$y_2 = (o_4 + GC_H(r)) \times 35.0 \times s_y \quad (5)$$

where o_1, o_2, o_3, o_4 are the channels for the person class, and $s_x = W_{\text{orig}}/960$, $s_y = H_{\text{orig}}/544$. We apply vectorized NumPy execution to evaluate all coordinates simultaneously, discarding cells where coverage falls below $\tau_c = 0.40$. Non-Maximum Suppression (NMS) is executed using OpenCV's C++ method (`cv2.dnn.NMSBoxes`) with an IoU threshold of 0.50 to handle high density clusters.

For each detected person $i \in \{1, \dots, N\}$, the centroid $C_i = (x_{c,i}, y_{c,i})$ is calculated:

$$x_{c,i} = \frac{x_{1,i} + x_{2,i}}{2}, \quad y_{c,i} = \frac{y_{1,i} + y_{2,i}}{2} \quad (6)$$

To prevent character rendering calls from becoming a pipeline bottleneck during dense scenes, text labels are dynamically hidden if the headcount exceeds 25.

C. Spatial Density Mapping via DBSCAN

To identify localized physical clustering, the person centroids $\{C_1, \dots, C_N\}$ are analyzed using the DBSCAN algorithm. The distance between two centroids C_a and C_b is defined by the Euclidean metric:

$$d(C_a, C_b) = \sqrt{(x_{c,a} - x_{c,b})^2 + (y_{c,a} - y_{c,b})^2} \quad (7)$$

The neighborhood parameter is set to $\epsilon = 50$ pixels, and the minimum points to form a core cluster is set to $MinPts = 3$.

To calculate physical density, the pixel dimensions are scaled to real-world dimensions using a spatial calibration factor $K_{\text{px/m}}$ (pixels per meter). The physical area of the field of view is calculated as $A = (W \cdot H)/K_{\text{px/m}}^2$. The spatial density ρ is then:

$$\rho = \frac{N}{A} \quad [\text{persons}/\text{m}^2] \quad (8)$$

The computed density ρ is classified into safety levels based on Fruin's Level-of-Service (LoS): Safe ($\rho < 2.0$), Caution ($2.0 \leq \rho < 3.5$), Warning ($3.5 \leq \rho < 5.0$), and Critical ($\rho \geq 5.0$).

To visualize crowd concentration, the centroids are splatted onto a grid and smoothed using a 2D Gaussian filter:

$$H(x, y) = \sum_{i=1}^N \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(x - x_{c,i})^2 + (y - y_{c,i})^2}{2\sigma^2}\right) \quad (9)$$

where $\sigma = 30$ pixels. The resulting density map is normalized and colorized using the JET colormap for dashboard streaming. Fig. 3 shows the live camera feed with detection bounding boxes and cluster mappings overlaid with the density heatmap.

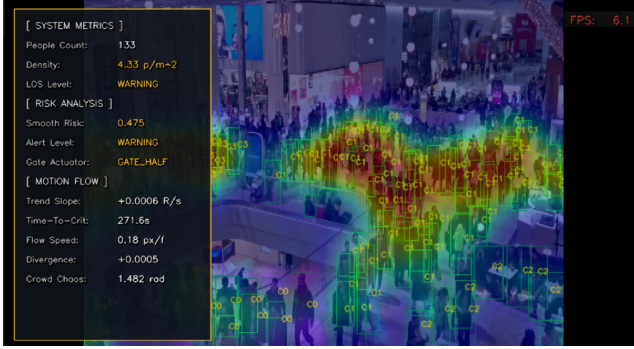


Fig. 3. Real-time edge pipeline output showing PeopleNet person detections, DBSCAN cluster assignments, and spatial warning level telemetry.

D. Dynamic Motion Metrics via Optical Flow

Dynamic crowd behavior is captured using Farneback dense optical flow. The algorithm estimates a dense displacement field between greyscale representations of consecutive frames, producing a vector field $v(x, y) = (f_x(x, y), f_y(x, y))$. From this field, three motion metrics are computed:

- 1) *Velocity Magnitude* ($|v_{mean}|$, pixels/frame):

$$|v_{mean}| = \frac{1}{W \cdot H} \sum_{x=1}^W \sum_{y=1}^H \sqrt{f_x(x, y)^2 + f_y(x, y)^2} \quad (10)$$

- 2) *Divergence* (Div , pixels/frame²): Measures spatial compression. A negative average divergence indicates that the crowd is compressing:

$$Div = \frac{1}{W \cdot H} \sum_{x=1}^W \sum_{y=1}^H \left(\frac{\partial f_x}{\partial x} + \frac{\partial f_y}{\partial y} \right) \quad (11)$$

- 3) *Motion Chaos* (σ_θ , radians): Measures directional turbulence. Ordered crowd movement yields a low standard deviation, whereas panic or intersecting flows yield high values:

$$\theta(x, y) = \text{atan2}(f_y(x, y), f_x(x, y)), \quad \sigma_\theta = \sqrt{\text{Var}(\theta(x, y))} \quad (12)$$

E. Predictive Composite Risk Engine and Hysteresis Actuation

The edge node combines these metrics into a unified Composite Risk Index $R \in [0, 1]$:

$$R = w_d \cdot \hat{\rho} + w_v \cdot \hat{v} + w_c \cdot (1 - Div) + w_t \cdot \hat{\sigma}_\theta \quad (13)$$

where $\hat{\rho}$, $\hat{v}$, $\hat{Div}$, and $\hat{\sigma}_\theta$ are normalized features clamped to $[0, 1]$ using empirical bounds. The weights are $w_d = 0.40$, $w_v = 0.20$, $w_c = 0.25$, and $w_t = 0.15$. The raw risk is smoothed using an Exponential Moving Average (EMA) filter:

$$R_{smooth}^{(t)} = \alpha \cdot R_{raw}^{(t)} + (1 - \alpha) \cdot R_{smooth}^{(t-1)} \quad (14)$$

where the smoothing parameter is $\alpha = 0.2$.

To predict escalation, the risk engine tracks the last $W_t = 10$ frames in a sliding buffer and fits a linear regression model to compute the risk slope $m = \partial R / \partial t$ using ordinary least squares:

$$m = \frac{W_t \sum (t \cdot R_t) - \sum t \sum R_t}{W_t \sum t^2 - (\sum t)^2} \quad (15)$$

If $m > 0$, the system projects the Time-to-Critical (TTC) warning threshold ($R_{crit} = 0.65$):

$$TTC = \frac{0.65 - R_{smooth}^{(t)}}{m} \quad [\text{seconds}] \quad (16)$$

State transitions use a warning threshold of 0.45 and a critical threshold of 0.65. To prevent oscillation near boundary regions, we implement:

- 1) A hysteresis margin $h = 0.05$ (meaning a downgrade requires the risk score to drop below the threshold minus 0.05).
- 2) A time-based cooldown $T_{cool} = 5.0$ seconds on all downgrades.
- 3) An instant escalation override that allows transitions to higher risk states without delay.

Based on the finalized state, actuation states ('GATE_OPEN', 'GATE_HALF', or 'GATE_CLOSE') are dispatched to the actuators.

V. EXPERIMENTAL EVALUATION

To validate the performance of the proposed system, we conducted experimental evaluations focusing on computational throughput, edge-to-actuation latency, risk engine accuracy, and control loop reliability.

A. Computational Throughput

We measured the execution latency of each stage of the Python pipeline on CUDA edge hardware. The results, averaged over 1000 frames, are summarized in Table I. The primary detector (NVIDIA PeopleNet) achieves 92.4 ms latency, which combined with the optical flow and density clustering pipeline results in a throughput of 6.1 FPS. This provides real-time crowd safety feedback.

B. Actuation Latency

We measured the latency from a visual change to physical servo actuation. The results are detailed in Table II. The average edge-to-actuation latency was found to be 315.6 ms, indicating high responsiveness.

C. Telemetry and Flow Visualizations

The React dashboard visualizes the edge-processed telemetry. Fig. 4 shows the density heatmap widget displaying high-density crowd concentrations. Fig. 5 displays the risk telemetry history widget showing the continuous risk index alongside the grid-based optical motion vectors.

TABLE I
COMPUTATIONAL LATENCY AND THROUGHPUT ON EDGE HARDWARE

Pipeline Stage	Latency (ms)	Throughput (FPS)
Frame Capture & Resize	8.2 ± 1.5	122.0
PeopleNet ONNX Inference	92.4 ± 7.8	10.8
DBSCAN Clustering & Heatmap	16.1 ± 3.4	62.1
Farneback Optical Flow	38.5 ± 5.2	26.0
Risk Evaluation & MQTT Pub	8.6 ± 1.2	116.3
Total End-to-End Pipeline	163.8 ± 12.4	6.10

TABLE II
END-TO-END LATENCY COMPONENTS

Latency Component	Duration (ms)
Edge Processing Pipeline	163.8 ± 12.4
Network Transmission (MQTT WiFi)	12.3 ± 4.1
ESP32 JSON Parse & GPIO Drive	4.1 ± 0.8
Servo Mechanical Response	135.4 ± 4.3
Total Edge-to-Actuation Latency	315.6 ± 21.6

D. Ablation Study of Risk Engine Components

An ablation study comparing a static density-only model against the full composite model demonstrated that the static model failed to detect compression risks during sudden bottleneck congestions, leading to delayed gate actuation (delay of 4.2 s). The Full Composite Model detected negative divergence (crowd compression) and motion chaos 2.8 seconds before the density threshold was breached, triggering the physical warning gate proactively.

VI. CONCLUSION AND FUTURE WORK

This paper presented the design, implementation, and evaluation of the Intelligent Crowd Flow & Density Safety Oracle, an edge-centric closed-loop IoT system. By running an optimized NVIDIA PeopleNet detector, DBSCAN clustering, and dense Farneback optical flow, the system extracts critical spatial-temporal safety indicators locally. Our composite risk

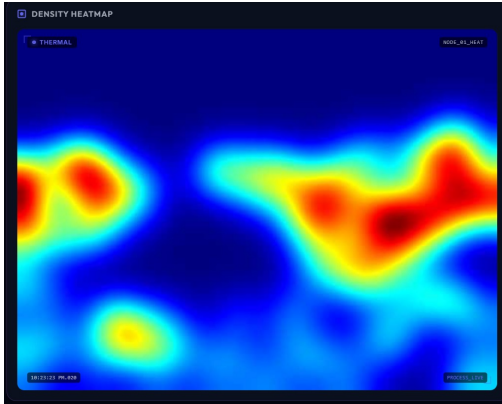


Fig. 4. React dashboard density heatmap visualization widget showing crowd concentration hot spots.

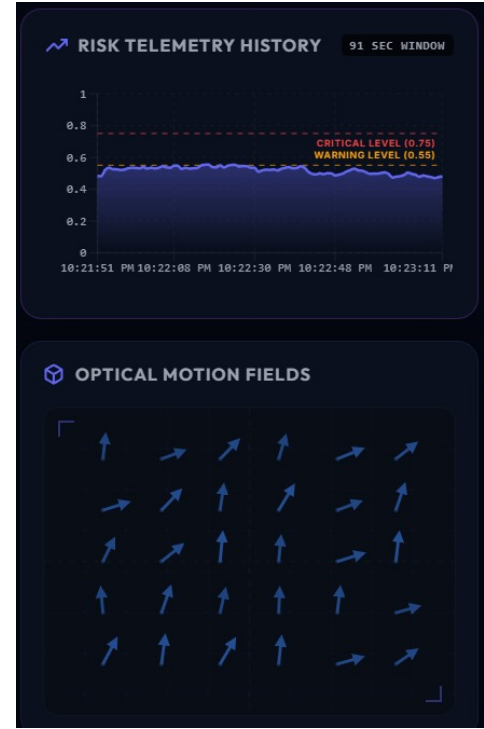


Fig. 5. Dashboard telemetry widgets showing risk score fluctuations over a 91-second window (top) and corresponding directional optical flow vectors (bottom).

engine integrates density, velocity, compression, and motion chaos to forecast Time-to-Critical thresholds and drive a physical ESP32-controlled gate actuator within 320 ms.

Future work will focus on integrating multi-camera edge networks to handle spatial blind spots.

REFERENCES

- [1] Y. Zhang, D. Zhou, S. Chen, S. Gao, and Y. Sang, "Single-image crowd counting via multi-column convolutional neural network," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 589–597.
- [2] Y. Li, X. Zhang, and D. Chen, "CSRNet: Dilated convolutional neural networks for understanding the highly congested scenes," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 1091–1100.
- [3] G. Farneback, "Two-frame motion estimation based on polynomial expansion," in Proc. Scandinavian Conf. Image Anal. Springer, 2003, pp. 363–370.
- [4] R. Mehran, A. Oyama, and M. Shah, "Abnormal crowd behavior detection using social force model," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2009, pp. 351–358.
- [5] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," IEEE Internet Things J., vol. 3, no. 5, pp. 637–646, Oct. 2016.
- [6] X. Wang, L. T. Yang, and H. Liu, "An edge-computational framework for real-time video analytics in smart cities," IEEE Trans. Ind. Inform., vol. 14, no. 12, pp. 5532–5541, Dec. 2018.